

IUT Nancy Charlemagne

Université de Lorraine

2 ter boulevard Charlemagne – BP 55227 – 54052 Nancy Cedex

Département informatique

Développement et mise en production d'une application de gestion de tâches hiérarchisées



Rapport de stage de BUT Informatique

Entreprise : Solco Etudiant : Florian HANNEZO

Tuteur IUT : Slim OUNI

Tuteur entreprise : Régis MARIQUE

Année universitaire 2025–2026

IUT Nancy Charlemagne

Université de Lorraine

2 ter boulevard Charlemagne – BP 55227 – 54052 Nancy Cedex

Département informatique

Développement et mise en production d'une application de gestion de tâches hiérarchisées



Rapport de stage de BUT Informatique

Entreprise : Solco Etudiant : Florian HANNEZO

Tuteur IUT : Slim OUNI

Tuteur entreprise : Régis MARIQUE

Année universitaire 2025–2026

Remerciements

Avant de commencer ce rapport, je tiens à remercier Solco pour m'avoir accueilli au sein de ses locaux et m'avoir offert la chance de travailler avec eux durant ces 8 semaines de stages.

Je remercie particulièrement Mr Régis Marique, mon directeur qui m'a offert cette opportunité de travailler dans les meilleures conditions au Luxembourg, en m'hébergeant et en m'accordant sa confiance tout le long de cette période.

Je remercie également toute l'équipe, qui a su m'accueillir de manière chaleureuse. Leur disponibilité et leur investissement m'ont permis de mieux comprendre les enjeux de mon travail et leur bienveillance ainsi que leur bonne humeur ont su sublimer mon séjour.

Ce stage a été une étape importante dans mon parcours, tant sur le plan professionnel que personnel. Je garderai un excellent souvenir de cette immersion au sein de votre structure.

Enfin, je souhaite exprimer ma gratitude à l'IUT Nancy Charlemagne et à ses enseignants, qui m'ont apporté les compétences et les connaissances nécessaires pour commencer ce stage avec toutes les ressources pour l'envisager.

Table des matières

1	Introduction	4
2	Présentation de l'entreprise	5
2.1	Secteur d'activité	5
2.2	Contexte de développement et enjeux de croissance	5
3	Contexte et enjeux du stage	6
3.1	Les ERP, des outils presque indispensables	6
3.2	Un enjeu technique pour Solco	6
3.3	Objectifs du Stage	7
4	Analyse, conception et choix des technologies utilisées	8
4.1	Étude de l'existant et analyse des besoins	8
4.2	Méthodologie de travail	9
4.3	Conception architecturale	9
4.3.1	Architecture globale (Client / Serveur)	9
4.3.2	Motif de conception : Architecture MVC	10
4.4	Modélisation de la base de données	10
4.5	Choix technologiques	12
5	Travaux réalisés	13
5.1	Construction du Backend	13
5.1.1	Base de données	13
5.1.2	Orm	14
5.1.3	Contrôleurs	17
5.1.4	Structuration d'un serveur d'application	22
5.2	Développement du Frontend	29
5.2.1	Le framework SB Admin 2	30
5.2.2	Réalisation des tableaux interactifs avec DataTables	30
5.2.3	Pages, Modals et Validation : Le lien Frontend-Backend	32
5.3	Déploiement sur un VPS	34
5.4	Phase de tests et validation des flux métiers	36
5.5	Documentation	37
5.5.1	Listing des fonctions et de leurs objectifs	37

5.5.2	Développement en local	38
5.5.3	Déploiement de l'application sur un serveur	38
6	Bilan personnel et professionnel	39
6.1	Bilan professionnel : De la théorie à la pratique	39
6.2	Bilan personnel : Donner du sens au code	39
6.3	Adéquation avec les compétences du BUT Informatique	40
	Conclusion	40
A	Diagramme de Gantt	44
B	Historique détaillé de l'entreprise Solco	45
	Fiche bibliothèque	46

Introduction

Du 16 février au 10 avril 2026, j'ai effectué un stage de fin d'études au sein de la société Solco, située à Livange (Luxembourg). Ma mission principale a consisté à concevoir et développer intégralement l'application web « Yieldtrack », un outil de gestion de projet (ERP) simplifié permettant de piloter l'activité par le « reste à faire ».

La structure de ce rapport reflète la progression logique de ce projet, de l'immersion métier jusqu'à la mise en production.

Dans un premier temps, j'aborderai le contexte de l'entreprise Solco et les enjeux stratégiques liés au développement d'un outil propriétaire. La deuxième partie sera consacrée à l'analyse des besoins et à la conception architecturale de l'application, justifiant le choix d'un environnement moderne basé sur Node.js et l'ORM Sequelize.

Le cœur de ce mémoire détaillera ensuite la réalisation technique des modules Backend et Frontend, ainsi que la méthodologie de déploiement sur un serveur VPS. Enfin, une analyse réflexive dressera le bilan de ces huit semaines, en évaluant l'adéquation entre les compétences acquises et les exigences du milieu professionnel.

Présentation de l'entreprise

Solco (également identifiée sous le nom de Solco Luxembourg) est une société de conseil luxembourgeoise spécialisée dans l'accompagnement des organisations vers l'excellence opérationnelle. La philosophie de l'entreprise repose sur une devise centrale : « Think Simple, Make Simple » (Penser simple, faire simple).

L'entreprise se positionne comme un partenaire stratégique pour les groupes établis qui souhaitent gagner en agilité. Elle opère principalement dans la région FranBenelux (France, Belgique, Luxembourg).

2.1 Secteur d'activité

Les services de Solco s'articulent autour de deux piliers complémentaires. Le premier concerne la simplification organisationnelle : il s'agit d'analyser et de restructurer les processus internes afin d'éliminer les tâches à faible valeur ajoutée et de fluidifier le quotidien des équipes. Le second pilier, centré sur l'engagement de projet orienté résultats, vise à accompagner les clients dans l'atteinte d'objectifs concrets grâce à une méthodologie axée sur la mobilisation des équipes et la mesure des résultats.

2.2 Contexte de développement et enjeux de croissance

Solco est une structure à taille humaine qui s'appuie sur une expertise consolidée dans le domaine du conseil en management. Bien que l'entreprise soit de création récente (2025), elle connaît une phase d'expansion rapide, illustrée par un portefeuille de clients s'étendant déjà sur la France, la Belgique et le Luxembourg .

Cette dynamique de croissance, marquée par un doublement attendu du nombre de collaborateurs et de projets, impose aujourd'hui une transition numérique majeure. Jusqu'alors, le suivi opérationnel reposait sur des outils artisanaux (fichiers Excel) qui atteignent désormais leurs limites en termes d'agilité et de fiabilité. Le projet Yieldtrack, objet de ce stage, s'inscrit précisément dans cette volonté de professionnalisation des outils internes pour soutenir le développement de la structure.

Note : Les détails chronologiques de la création de l'entreprise sont dans l'Annexe.

Contexte et enjeux du stage

3.1 Les ERP, des outils presque indispensables

Un ERP (Enterprise Resource Planning) est un progiciel de gestion intégré qui permet de centraliser l'ensemble des données et des processus d'une entreprise au sein d'une base de données unique. Dans le monde professionnel actuel, ces outils sont devenus indispensables à plusieurs égards. En premier lieu, ils assurent la centralisation et l'intégrité des données en évitant la dispersion des informations dans des fichiers Excel disparates. Chaque collaborateur travaille ainsi sur une version unique et vérifiée. Ils permettent également d'optimiser les processus en automatisant certaines tâches répétitives, comme le changement de statut d'une tâche, réduisant ainsi le risque d'erreur humaine. Enfin, grâce à leurs modules de reporting, les ERP offrent une aide à la décision en temps réel, donnant à la direction une visibilité permanente sur l'avancement des projets et les difficultés rencontrées.

3.2 Un enjeu technique pour Solco

Pour Solco, entreprise de conseil, le développement de son propre ERP, YieldTrack, représente un enjeu stratégique et technique majeur. L'objectif n'est pas seulement de disposer d'un outil interne, mais de proposer une solution propriétaire à forte valeur ajoutée pour ses clients.

- **Différenciation par l'innovation technique :** Contrairement aux outils de gestion de tâches classiques, Solco a fait le choix technique de piloter les projets par le Remaining Work (RW) ou "Reste à faire". Cela impose une architecture robuste capable de gérer un historique précis des saisies.

Face aux grands ERP, que l'on peut retrouver sur le marché, ici le but est d'avoir une interface et un fonctionnement plus simple. En se concentrant sur des fonctionnalités essentielles, Solco veut offrir une prise en main des plus efficaces pour l'utilisateur.

- **Personnalisation et adaptabilité :** Développer un ERP en interne permet à Solco de choisir d'intégrer de nouvelles fonctionnalités à l'ensemble de ses clients, mais aussi aux clients avec des besoins spécifiques (versioning).

L'enjeu technique consiste donc à transformer une expertise métier en un produit logiciel.

3.3 Objectifs du Stage

L'objectif de mon stage a été la création de cet ERP from scratch (de 0), nommé Yield-track.

La mission se déclinait en trois grands axes. Le premier concernait le développement de l'application elle-même : il s'agissait de concevoir et structurer le backend selon une logique d'ERP simplifié, puis d'adapter un frontend existant pour obtenir une interface graphique claire et fonctionnelle. Le deuxième axe portait sur le déploiement et la gestion de l'application, avec la mise en place d'un dépôt Git pour le versioning et la mise en ligne sur un serveur de production. Enfin, le troisième axe était consacré à la documentation : définir et localiser les fonctions du code, puis rédiger des tutoriels de déploiement destinés à faciliter la reprise du projet.

Au-delà de l'aspect technique, ce stage avait pour but de me faire comprendre comment traduire des besoins métiers complexes en solutions logicielles concrètes au sein d'une structure de conseil.

Analyse, conception et choix des technologies utilisées

4.1 Étude de l'existant et analyse des besoins

Le contexte et l'existant

Le projet YieldTrack s'inscrit dans une volonté d'optimisation et de modernisation des outils de suivi. Jusqu'à présent, Solco mettait en place des fichiers Excel chez ses clients pour gérer le suivi de projet. Bien que ces fichiers soient automatisés à l'aide de macros, la saisie y reste souvent longue et peu intuitive. L'objectif principal de cette nouvelle application est donc de remplacer ces tableurs par une interface web plus ergonomique, rapide et uniformisée (tout affichage devant reposer sur une structure commune en tableau), tout en conservant la possibilité d'exporter les données pour les utilisateurs nécessitant un traitement externe.

Les acteurs du système

Pour répondre aux besoins métiers, l'application s'adresse à différents types d'utilisateurs. Le système de droits repose sur plusieurs profils (un même acteur pouvant posséder plusieurs rôles simultanément) :

- **Administrateur & Key User** : Chargés de l'administration de l'application et de l'attribution des rôles.
- **Program Manager** : Responsable de la création et de la gestion globale des projets.
- **Project Leader** : En charge de la structuration des projets en lots de travaux (*Work Packages*) et de la création des tâches associées.
- **Team Leader** : Affecte les tâches de son unité (département) aux contributeurs appropriés au sein de cette même unité.
- **Contributeur** : Intervient en bout de chaîne pour déclarer son « reste à faire » (*Remaining Work*) sur les tâches qui lui sont assignées.

4.2 Méthodologie de travail

Pour mener à bien le développement de cette application, j'ai adopté une méthodologie itérative. J'ai mis en place une liste de tâches (*To-Do list*) que je mettais à jour et communiquais régulièrement à mon tuteur.

Dans un premier temps, mes efforts se sont concentrés sur l'apprentissage et la mise en place des fondations techniques du projet (prise en main de l'ORM, configuration de base du serveur Express). Une fois ce socle consolidé, le développement a été guidé par une liste de fonctionnalités prioritaires classées par ordre d'importance logique (et non par vue ou par page). Enfin, les itérations suivantes ont été consacrées au débogage et à l'application de correctifs (*fix*) pour stabiliser l'outil.

4.3 Conception architecturale

4.3.1 Architecture globale (Client / Serveur)

L'application YieldTrack repose sur une architecture Client/Serveur classique. Le client (navigateur web) se charge de l'affichage de l'interface, tandis que le serveur héberge la logique métier et communique avec la base de données.

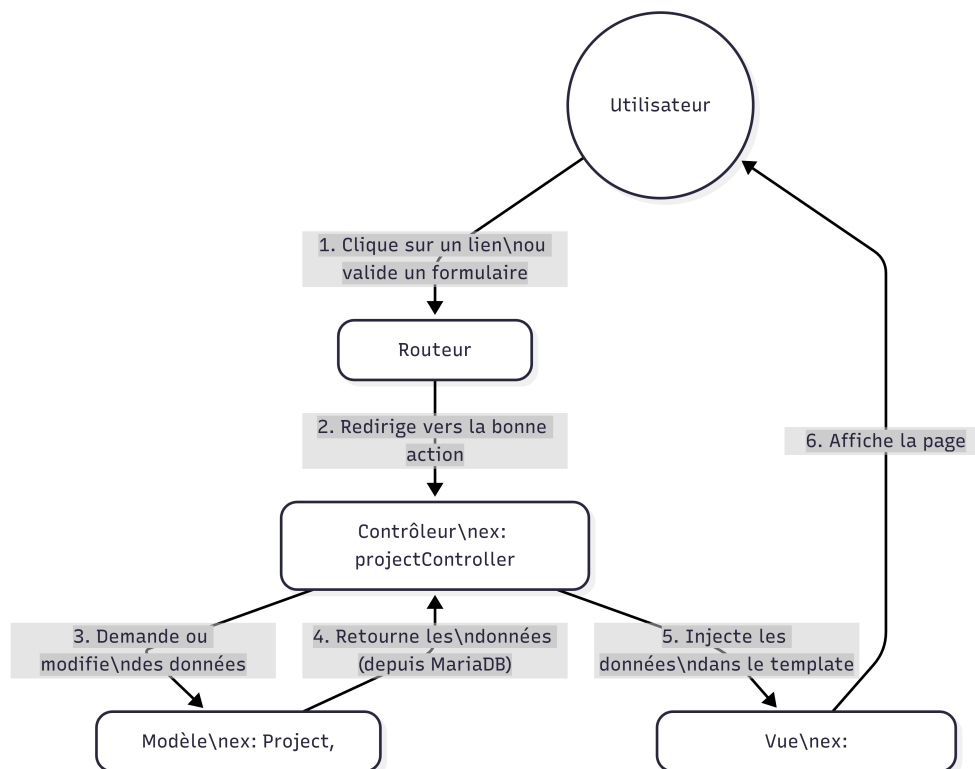


FIGURE 4.1 – Architecture globale Client/Serveur de YieldTrack

4.3.2 Motif de conception : Architecture MVC

Pour structurer le code, j'ai choisi le patron de conception **MVC (Modèle-Vue-Contrôleur)**. Ce choix permet de séparer clairement les responsabilités :

- **Le Modèle** : Gère la structure des données et la communication avec la base de données.
- **La Vue** : Représente l'interface graphique envoyée à l'utilisateur.
- **Le Contrôleur** : Intercepte les requêtes de l'utilisateur, demande les données au Modèle, et les injecte dans la Vue correspondante.

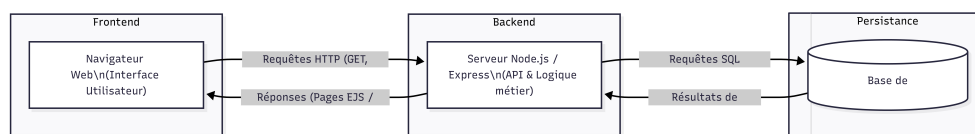


FIGURE 4.2 – Flux de traitement d'une requête selon le patron MVC

4.4 Modélisation de la base de données

La structuration des données est au cœur de YieldTrack. Nous avons conçu une base de données relationnelle autour des entités du projet. La gestion est hiérarchique : un Projet (Projects) est découpé en lots de travaux (WorkPackages), eux-mêmes divisés en Tâches (Tasks). Enfin, les utilisateurs déclarent leur temps restant (Rws) sur ces tâches.

NB : Ce modèle conceptuel de données (MCD) est très simplifié afin de ne pas vous perdre.

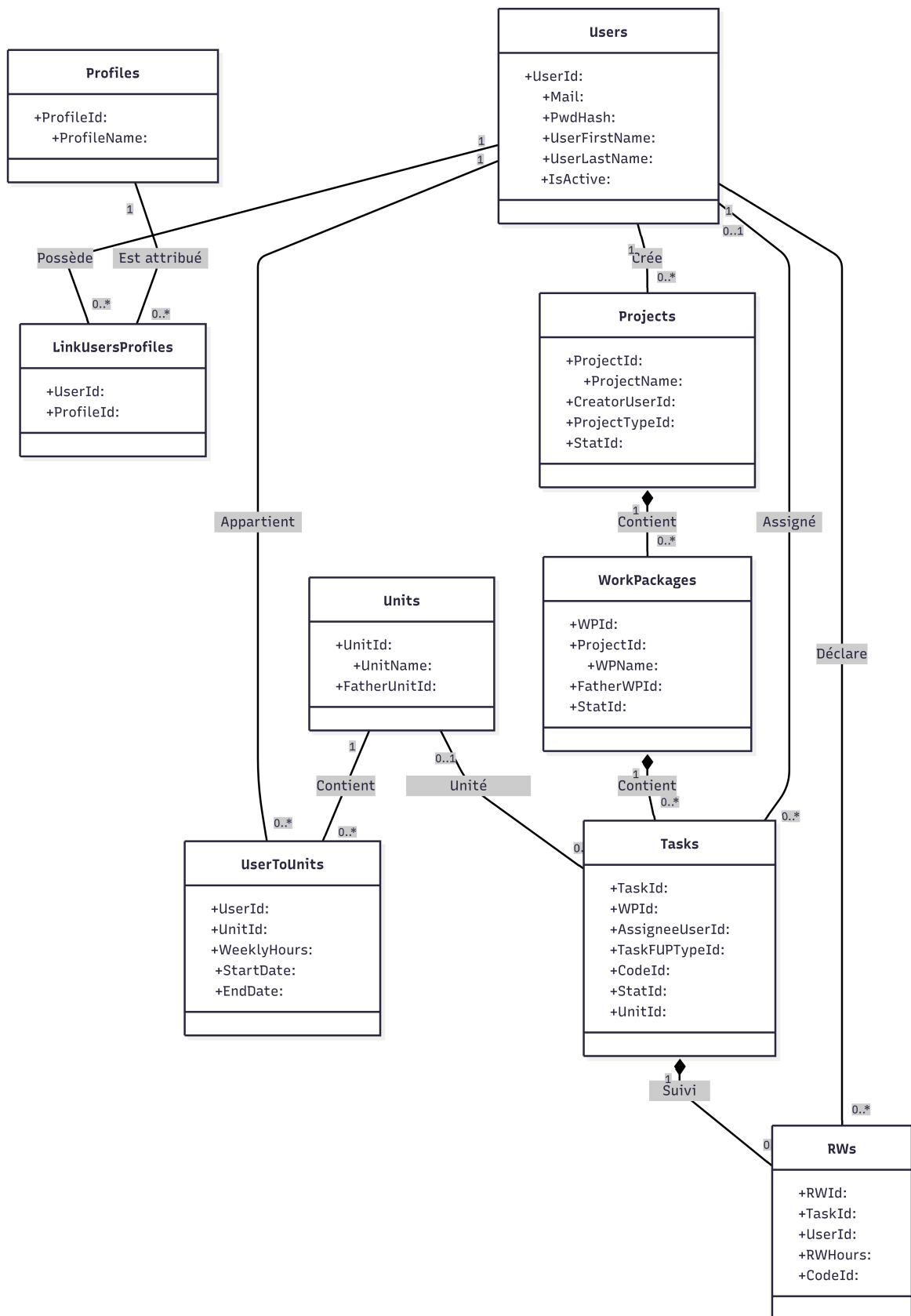


FIGURE 4.3 – MCD simplifié illustrant les entités principales de l’application

4.5 Choix technologiques

Dès le début du stage, mon tuteur m’a détaillé les attentes précises et les fonctionnalités facultatives (les « *nice to have* »). Pour répondre à ces exigences matérielles et architecturales, ensemble, nous avons réalisé un comparatif pour nous orienter vers des solutions modernes, performantes et libres de droits (*open source*).

- **Backend et Serveur (Node.js & Express) :** Lors de notre réflexion, nous avons hésité entre PHP et Node.js. Nous avons finalement retenu **Node.js** pour sa légèreté et sa modernité. Pour l’accompagner, le framework **Express** s’est imposé naturellement grâce à sa simplicité et son approche minimaliste.
- **Sécurisation (TypeScript) :** Bien que mes connaissances initiales en JavaScript fussent limitées (ce qui représentait une belle opportunité de progression), j’ai choisi d’utiliser **TypeScript** pour la construction du backend. Son avantage majeur est la réintroduction d’un typage rigoureux, permettant d’éviter de nombreuses erreurs de développement et d’améliorer la sécurité, une approche qui rappelle le langage Java étudié à l’IUT.
- **Interaction avec la base de données (Sequelize) :** N’ayant aucune expérience préalable avec les ORM (*Object-Relational Mapping*), nous avons sélectionné **Sequelize**. D’après nos recherches, c’est l’outil le plus recommandé et le plus simple à prendre en main pour interagir avec une base de données sous Node.js.
- **Système de gestion de base de données (MariaDB) :** Le choix s’est porté sur **MariaDB** plutôt que MySQL. En plus d’être une technologie robuste que j’avais déjà pu étudier en cours, elle offre l’avantage d’être totalement gratuite et communautaire, évitant ainsi tout risque de surcoût de licence à l’avenir.
- **Interface graphique (SB Admin 2) :** Pour la partie visuelle (Frontend), mon tuteur m’a orienté vers **SB Admin 2**, un framework basé sur Bootstrap 4. C’est un choix très pertinent car il offre des composants classiques, sobres et simples à déployer, parfaits pour bâtir une interface administrative de type tableau de bord.

Travaux réalisés

5.1 Construction du Backend

Bien que ce rapport présente le développement technique de manière séquentielle pour des raisons de clarté, la réalité du projet fut beaucoup plus itérative. La construction du Backend et le développement de l'interface graphique (Frontend) ont avancé presque simultanément. La création des flux utilisateurs (workflows) et les échanges réguliers avec mon tuteur ont mis en évidence de nouveaux besoins métiers. Ainsi, l'architecture du Backend n'a pas été figée dès le premier jour, mais a su évoluer et s'enrichir tout au long de mon stage pour répondre précisément aux attentes fonctionnelles.

5.1.1 Base de données

Initialement, mon tuteur m'a transmis l'architecture globale des données sous la forme d'une liste structurée détaillant les tables, les colonnes et les types de données attendus. À partir de cette base, j'ai pris la responsabilité de l'adapter aux contraintes techniques du projet et de la traduire concrètement en un script SQL de création complet (`creation_db.sql`). Dans la foulée, j'ai élaboré un script d'insertion de données fictives afin de pouvoir simuler et tester concrètement les différents flux de l'application selon les profils (point de vue administrateur, contributeur, *project leader*, etc.).

Cependant, la confrontation de ce modèle théorique à la réalité du développement (notamment lors de la création des vues et des contrôleurs) m'a conduit à faire évoluer significativement cette structure initiale. En échangeant avec mon tuteur et en analysant les parcours utilisateurs, j'ai identifié plusieurs axes d'amélioration qui ont donné lieu à des modifications importantes de la base de données :

— 1. L'ajout d'un niveau hiérarchique supérieur : Les Projets

La modification la plus structurante a été l'introduction de la table `Projects`. À l'origine, l'entité la plus haute était le lot de travaux (`WorkPackages`). J'ai restructuré ce modèle pour que chaque `WorkPackage` soit rattaché à un projet parent. Conséquence logique de ce changement, la notion de "type" a été remontée au niveau du projet (la table `WPTypes` a été supprimée au profit de `ProjectTypes`). Cette refonte globale de la hiérarchie a été décidée, car elle simplifie et unifie l'utilisation des types. Ainsi les `WorkPackages` d'un projet "Corporate Template" sont donc tous du même type. Cela

suit également la logique de laisser le choix du type uniquement au Program Manager.

— 2. La centralisation et fiabilisation des statuts

Dans la version initiale, les statuts des tâches ou des lots étaient gérés par de simples chaînes de caractères (*VARCHAR*). J'ai pris l'initiative de créer une table de référence indépendante nommée *Status*. Désormais, les tables *Projects*, *WorkPackages* et *Tasks* pointent toutes vers cette même table via une clé étrangère (*StatId*). Ce choix architectural m'a permis d'unifier l'affichage frontend avec une liste déroulante fixe, pour éviter également les fautes de frappe utilisateur.

— 3. L'ajustement des saisies (RWs)

En affinant les cas d'usage du "Reste à faire" (*Remaining Work*), nous avons revu l'unité de mesure globale : les budgets et saisies sont passés des minutes aux heures (*TaskBudgetHours*, *RWHours*). Surtout, j'ai modifié la table *RWs* en passant la date au format *DATETIME* et en supprimant la contrainte qui bloquait les doubles saisies le même jour. J'y ai également ajouté la possibilité de lier un help code (*CodeId*) et un commentaire texte. Ces évolutions offrent une plus grande souplesse aux contributeurs, car la double saisie est un aspect obligatoire. Le help code et le commentaire permettent un suivi et une compréhension accrue pour pouvoir aider le contributeur à réaliser son travail.

— 4. L'optimisation de l'attribution des ressources

Enfin, la gestion des droits et des affectations a été épurée. J'ai remplacé la table complexe *WPContributors* par une table de liaison simplifiée *ProjectLeaders*, qui associe directement un utilisateur à un projet sans notion de profil redondante. De plus, au niveau des tâches (*Tasks*), j'ai rendu l'assignation d'un utilisateur optionnelle (champ *NULL* autorisé) et nous avons fait le choix de supprimer la table *Absences* qui était initialement prévue. Ce changement est dû à la philosophie d'association que j'avais mal comprise. En effet, une tâche qui vient d'être créée possède une unité, mais pas d'utilisateur assigné. C'est le team leader avec cette unité qui va devoir l'affilier à un contributeur de cette même unité.

Ces nombreuses itérations sur le schéma SQL ont été fondamentales : elles m'ont assuré de construire des modèles Sequelize robustes et parfaitement alignés avec les besoins fonctionnels finaux de YieldTrack.

5.1.2 Orm

Pour faire communiquer la base de données MariaDB avec le serveur Node.js, le choix technologique s'est porté sur l'utilisation d'un ORM (*Object-Relational Mapping*), et plus spécifiquement **Sequelize**.

De manière théorique, un ORM est une couche d'abstraction logicielle qui sert d'interface entre un programme orienté objet (ici le code JavaScript/TypeScript) et une base de données relationnelle. Au lieu de rédiger des requêtes SQL textuelles brutes (SELECT, INSERT, JOIN) qui peuvent être difficiles à maintenir et vulnérables aux failles de sécurité (comme les injections SQL), l'ORM permet de manipuler les tables de la base de données comme s'il s'agissait de simples objets informatiques.

Dans le cadre du projet YieldTrack, cette technologie s'est révélée absolument essentielle. L'application gérant une structure de données complexe et très hiérarchisée (Projets, Lots de travaux, Tâches, Saisies de temps, Utilisateurs et Départements), il aurait été très risqué de gérer manuellement les centaines de requêtes SQL nécessaires à son fonctionnement. L'ORM agit comme un garant de l'intégrité des données : il vérifie le type et le format de chaque information avant même de l'envoyer à la base de données.

Le défi de la "traduction" et les incohérences de nommage La première étape de mise en place a consisté à "traduire" mon script SQL de création de base de données (`creation_db.sql`) en modèles Sequelize (des classes TypeScript). C'est lors de cette phase que j'ai rencontré mes plus grandes difficultés techniques.

Le principal obstacle a été la correspondance (le *mapping*) entre le nom des colonnes dans la base de données et le nom des champs dans mon code. Dans mon script SQL, les colonnes avaient des noms spécifiques (par exemple `UserId`, `TaskBudgetHours`, etc.). Or, lors de la déclaration dans l'ORM, j'ai été confronté à de nombreuses incohérences et erreurs de requêtes causées par des différences de conventions de nommage ou des oublis de configuration. Sequelize s'attend parfois par défaut à des noms de colonnes formatés différemment (en *camelCase* ou *snake_case*). Il a donc fallu que je configure rigoureusement chaque modèle pour lier explicitement la propriété de l'objet à la colonne exacte en base (via l'attribut `field`).

La barrière de la syntaxe TypeScript À ce défi de configuration s'est ajoutée ma méconnaissance initiale de la syntaxe avancée de TypeScript. TypeScript impose une rigueur extrême via le typage statique. Pour définir un modèle Sequelize fonctionnel, il ne suffit pas de créer une simple classe ; il faut déclarer des interfaces complexes définissant les attributs de création (ce qui est requis pour créer une ligne) et les attributs du modèle (ce qui est retourné par la base), puis étendre (*extends*) correctement la classe `Model` de Sequelize. Ne maîtrisant pas encore parfaitement ces concepts, j'ai mis un temps considérable à comprendre la structure exigée par l'outil et à résoudre les erreurs de compilation qui bloquaient le démarrage du serveur.

Un déclic salvateur pour la suite du projet Cependant, cet investissement en temps pour comprendre et configurer l'ORM a été l'un des moments les plus décisifs de mon stage. Une fois la syntaxe de TypeScript assimilée et les modèles correctement liés entre eux (grâce aux associations `hasMany`, `belongsTo`, etc.), j'ai eu un véritable déclic.

La manipulation des données est soudainement devenue extrêmement fluide. Récupérer un projet avec toutes ses tâches associées, ou calculer les heures restantes d'un utilisateur, ne prenait plus que quelques lignes de code très lisibles. Cette prise en main réussie a totalement débloqué l'avancement de mon projet : elle m'a permis de développer les contrôleurs du Backend beaucoup plus rapidement et d'alimenter mes vues Frontend avec des données structurées et fiables. L'ORM, initialement perçu comme une contrainte complexe, est devenu mon meilleur allié pour la suite du développement.

Structure d'un modèle Sequelize Pour illustrer la mise en œuvre de l'ORM, voici un exemple simplifié basé sur le modèle `Status`. Ce code montre comment est définie la structure de l'objet côté serveur tout en précisant le mappage exact avec la table physique en base de données :

```
1 // Définition de la classe pour le typage TypeScript
2 class Status extends Model {
3   public StatId!: number;
4   public StatName!: string;
5 }
6
7 // Initialisation du mapping avec la base de données
8 Status.init({
9   StatId: {
10     type: DataTypes.INTEGER.UNSIGNED,
11     autoIncrement: true,
12     primaryKey: true,
13     field: 'StatId' // Lien explicite avec la colonne SQL
14   },
15   StatName: {
16     type: DataTypes.STRING(100),
17     allowNull: false,
18     field: 'StatName'
19   }
20 }, {
21   sequelize,
22   tableName: 'Status', // Nom de la table dans MariaDB
23   timestamps: false
24 });
```

Listing 5.1 – Exemple de modèle Sequelize en TypeScript

Comme nous pouvons le constater, chaque attribut doit être rigoureusement défini. L'utilisation de la propriété `field` a été cruciale pour résoudre les incohérences de nommage que j'ai rencontrées entre les conventions de la base de données et celles du code TypeScript.

Simplification des requêtes : Comparaison SQL vs Sequelize L'intérêt majeur de l'ORM réside dans sa capacité à abstraire des opérations complexes, comme les jointures. À titre de comparaison, voici la différence entre une requête SQL classique et son équivalent via Sequelize pour récupérer un projet spécifique avec l'intégralité de ses lots de travaux (WorkPackages) :

— Approche en SQL traditionnel :

```
1  SELECT * FROM Projects p
2  LEFT JOIN WorkPackages wp ON p.ProjectId = wp.ProjectId
3  WHERE p.ProjectId = 1;
```

— Approche avec Sequelize (TypeScript) :

```
1  const project = await Project.findByPk(1, {
2      include: [WorkPackage]
3  });
```

Cette comparaison met en évidence pourquoi l'ORM est devenu indispensable au projet :

1. **Lisibilité** : Le code est beaucoup plus concis et exprime l'intention métier ("trouver un projet et inclure ses lots") plutôt que la mécanique technique des jointures.
2. **Sécurité** : Sequelize utilise automatiquement des requêtes préparées, ce qui protège nativement l'application contre les injections SQL sans effort supplémentaire de ma part.
3. **Typage** : Grâce à TypeScript, l'objet `project` retourné est immédiatement reconnu par l'éditeur de code, ce qui m'a permis d'accéder aux propriétés des lots de travaux avec l'autocomplétion, réduisant ainsi drastiquement le risque d'erreurs de développement.

5.1.3 Contrôleurs

Si les modèles définissent la structure des données de YieldTrack, les contrôleurs en constituent le véritable « cerveau ». Dans l'architecture MVC (Modèle-Vue-Contrôleur),

le contrôleur agit comme un chef d'orchestre : il intercepte les requêtes HTTP de l'utilisateur, interroge l'ORM Sequelize pour récupérer ou modifier les données, applique les règles métier spécifiques à Solco, et retourne enfin la réponse appropriée à l'interface graphique.

Cette étape du développement a été la plus stimulante et la plus gratifiante de mon stage. C'est ici que l'application prend vie, transformant des tables de données statiques en un outil de gestion dynamique et intelligent. Afin de présenter ce travail de manière pertinente, j'ai choisi de structurer l'explication de mes contrôleurs par thématiques, en suivant une logique de complexité croissante.

Sécurité et gestion administrative

La sécurité constitue la première ligne de défense de l'ERP. Cette logique est principalement centralisée dans le fichier `authControllers.ts`. Lorsqu'un utilisateur tente de se connecter, le contrôleur vérifie son adresse e-mail puis compare son mot de passe.

C'est à cette étape qu'intervient une notion fondamentale étudiée à l'IUT : le **hachage** (*hashing*). Dans la base de données, les mots de passe sont transformés en empreintes indéchiffrables via la bibliothèque `bcrypt`. Le contrôleur ne "déchiffre" pas le mot de passe ; il hache la saisie de l'utilisateur et la compare à l'empreinte stockée, comme l'illustre cet extrait :

```
1 // 1. Recherche de l'utilisateur par son email
2 const user = await User.findOne({
3   where: { email },
4   include: ['profiles']
5 });
6
7 if (!user) return res.render('login', { error: 'Unknown email.'
8   });
9
10 // 2. Comparaison sécurisée des empreintes (Hachage)
11 if (!await bcrypt.compare(password, user.pwd)) {
12   return res.render('login', { error: 'Password incorrect.'
13     });
14 }
15
16 // 3. Mise à jour de la date de dernière connexion
17 await user.update({ LastLoginAt: new Date() });
```

Listing 5.2 – Logique de vérification du mot de passe dans `authControllers.ts`

En complément de la sécurité, j'ai développé des contrôleurs d'administration (`UserController.ts`,

unitsController.ts) pour gérer les opérations CRUD (*Create, Read, Update, Delete*). J'ai également mis en place un module de récupération de mot de passe via e-mail avec nodemailer.

Pour des raisons de sécurité, les identifiants SMTP ne sont pas écrits "en dur" mais isolés dans un fichier de variables d'environnement (.env) :

```
1 const transporter = nodemailer.createTransport({
2   host: process.env.SMTP_HOST,
3   port: Number(process.env.SMTP_PORT) || 587,
4   secure: process.env.SMTP_PORT === '465',
5   auth: {
6     user: process.env.SMTP_USER,
7     pass: process.env.SMTP_PASSWORD
8   },
9 });
```

Listing 5.3 – Configuration du transporteur Nodemailer via le fichier .env

Bien que cette logique applicative soit fonctionnelle, j'ai rencontré une limite technique liée à la délivrabilité : les e-mails atterrissent souvent dans les *spams*. En utilisant la plateforme *mail-tester.com*, j'ai compris que le problème provenait de la configuration DNS du nom de domaine (absence d'enregistrements SPF/DKIM), une optimisation d'infrastructure que je n'ai pas eu le temps d'implémenter.

Gestion de la hiérarchie et logique métier complexe

La véritable valeur ajoutée de YieldTrack réside dans sa gestion de l'arborescence (Projets → Work Packages → Tâches) et du suivi du temps. Pour en saisir la complexité, je me suis rendu chez un client de Solco, l'entreprise SEO (*Société Électrique de l'Our*).

Cette immersion terrain m'a permis d'observer l'utilisation de leurs fichiers Excel : lenteur de saisie, écrasement de formules, et impossibilité de consolider les données. Elle m'a surtout permis de clarifier la définition du *Remaining Work* (RW). Contrairement au temps "passé", le RW est **l'estimation du temps restant** avant la fin de la tâche. Pour pallier les limites d'Excel, j'ai conçu le tasksController.ts de façon à ce qu'il enregistre chaque nouvelle estimation comme une nouvelle ligne horodatée en base de données, offrant ainsi un historique complet de l'avancement.

L'affichage de ces données a soulevé un défi architectural. Pour afficher une tâche, il faut également récupérer les informations de son *Work Package* et de son Projet. J'ai dû exploiter les requêtes imbriquées avec Sequelize (propriété `include`). La documentation officielle s'étant avérée insuffisante pour gérer les alias (`as`) et le `strict` type, je me suis appuyé sur

Stack Overflow et des outils d'I.A. pour déconstruire la syntaxe exigée :

```
1 // Extraction de la hiérarchie complète d'un projet
2 const project = await Project.findByPk(projectId, {
3   include: [{
4     model: WorkPackage,
5     as: 'workPackages',
6     include: [{
7       model: Task,
8       as: 'tasks',
9       include: [{
10        model: User,
11        as: 'assignee', // Jointure pour afficher le
12                        // nom du contributeur
13        attributes: ['UserFirstName', 'UserLastName']
14      }]
15    }]
16  }]);
```

Listing 5.4 – Exemple de requête imbriquée avec Sequelize dans un contrôleur

Fonctionnalités avancées : Automatisation, droits et exportation

La dernière thématique illustre ma capacité à implémenter des règles métier spécifiques et à manipuler des données de masse.

A. Le système de Templates et la gestion des droits

L'application permet de générer des arborescences de projets à partir de modèles (*Templates*). Durant le développement, nous avons pris la décision de supprimer la notion de type au niveau du *Work Package* pour la remonter au niveau du Projet (*ProjectTypes*). Cela a déplacé la complexité vers les contrôleurs (*wpControllers.ts* et *projectController.ts*) pour la gestion des droits.

Pour les *Corporate Templates* (un modèle de projet standardisé propre à l'entreprise), j'ai dû coder une vérification asymétrique : ils sont utilisables par tous, mais modifiables **uniquement** par les collaborateurs possédant un double profil (*Project Leader* ET *Program Manager*).

```
1 const isProjectLeader = userProfiles.includes(4);
2 const isProgramManager = userProfiles.includes(3);
3
4 if (project.typeId === CORPORATE_TEMPLATE_ID) {
5   if (!(isProjectLeader && isProgramManager)) {
```

```

6         return res.status(403).send("Modification reserved for
           PL & PM profiles.");
7     }
8 }

```

Listing 5.5 – Logique de vérification du double profil (PL + PM)

Cette logique m’a fait comprendre qu’un ERP est le véritable garant des processus hiérarchiques d’une entreprise.

B. L’exportation de données (CSV)

Certains utilisateurs ayant besoin de *reportings* externes, j’ai développé le `exportController.ts`. C’est ici que Sequelize s’est avéré redoutable : grâce au *Eager Loading* (chargement anticipé / immédiat), j’ai pu récupérer des milliers de tâches avec leurs hiérarchies en une seule requête optimisée, évitant une cascade de jointures SQL illisibles :

```

1 const exportData = await Task.findAll({
2     include: [
3         { model: WorkPackage, include: [{ model: Project,
4           include: [ProjectType, Status] }] },
5         { model: User, as: 'assignee', include: [{ model: Unit
6           }] },
7         { model: RWs, limit: 1, order: [['RWDate', 'DESC']] }
           // Dernier RW
8     ]
9 });

```

Listing 5.6 – Exemple conceptuel de la requête d’export avec Sequelize

Mon contrôleur se charge ensuite d’itérer sur ce flux de données tridimensionnel pour l’"aplatir" (*mapping*), formater les dates, et générer un fichier CSV plat et téléchargeable.

Cartographie exhaustive des Contrôleurs Afin de documenter précisément l’architecture logicielle de mon travail, voici la liste des contrôleurs développés ainsi que leurs fonctions principales :

- `authControllers.ts` :
 - `login` : Vérification des identifiants, hachage bcrypt et génération du token JWT.
 - `renderResetPassword / resetPassword` : Interface et logique de réinitialisation sécurisée.
 - `logout` : Destruction de la session.

- `UserController.ts` & `unitsController.ts` :
 - `getAll... / get...ById` : Affichage des listes et détails des collaborateurs et départements.
 - `create... / update... / delete...` : Opérations CRUD d'administration.
- `adminResourcesController.ts` :
 - Gère les opérations de création et de mise à jour des ressources annexes (Codes analytiques, Profils, Statuts).
- `projectController.ts` :
 - `getProjects` : Affichage filtré des projets (Production, Templates, Corporate) selon les droits de l'utilisateur.
 - `getProjectDetails` : Requête imbriquée majeure affichant l'arborescence complète.
 - `createProject / updateProject` : Logique de création avec affectation du *Project Leader*.
- `wpControllers.ts` :
 - `createWP / updateWP` : Gestion des lots de travaux.
 - `generateFromTemplate` : Logique d'automatisation dupliquant l'arborescence d'un modèle vers un nouveau projet.
- `tasksController.ts` :
 - `createTask / updateTask` : Attribution des tâches aux contributeurs et aux unités.
 - `addRW` : Insertion d'une nouvelle estimation de *Remaining Work* avec historisation.
 - `archiveTask` : Logique de clôture désactivant la tâche sans la supprimer de la base.
- `exportController.ts` :
 - `exportData` : Agrégation des données massives via l'ORM et formatage à la volée en fichier CSV.

5.1.4 Structuration d'un serveur d'application

Maintenant que la base de données est modélisée et que la logique métier est encapsulée dans les contrôleurs, il est nécessaire de mettre en place l'infrastructure logicielle capable de recevoir les requêtes des utilisateurs et de leur distribuer les bonnes réponses. C'est le rôle du serveur d'application.

Node.js et Express

Définitions et rôles Le socle technique de mon Backend repose sur le couple **Node.js** et **Express**. Node.js est un environnement d'exécution asynchrone qui permet d'utiliser le langage JavaScript (et par extension TypeScript) côté serveur, en dehors d'un navigateur web. Son architecture orientée événements le rend particulièrement performant pour gérer un grand nombre de connexions simultanées.

Cependant, Node.js brut nécessite beaucoup de code pour gérer des tâches web basiques. C'est ici qu'intervient **Express**. Il s'agit du *framework* (cadre de travail) le plus populaire pour Node.js. Minimaliste et flexible, Express fournit une syntaxe simple et robuste pour créer le serveur, gérer le routage des URL et intercepter les requêtes HTTP (GET, POST, etc.).

Explication et enjeux du fichier `app.ts` Le fichier `app.ts` est la pierre angulaire de l'application : c'est le point d'entrée du serveur. Son enjeu principal est la configuration globale. C'est dans ce fichier que j'initialise l'application Express, que je définis le moteur de rendu graphique (EJS), que je configure l'accès aux fichiers statiques (CSS, images) et, surtout, que j'aiguille les requêtes vers les bons fichiers de routes.

```
1 import express from 'express';
2 import cookieParser from 'cookie-parser';
3 import projectRoutes from './routes/projectRoutes';
4
5 const app = express();
6
7 // Configuration du moteur de vues (Frontend)
8 app.set('view engine', 'ejs');
9 app.set('views', path.join(__dirname, '../views'));
10
11 // Middleware pour parser les requêtes et lire les cookies
12 app.use(express.json());
13 app.use(express.urlencoded({ extended: true }));
14 app.use(cookieParser()); // Essentiel pour la gestion des
    sessions
15
16 // Déclaration des routes
17 app.use('/projects', projectRoutes);
18 // ... autres routes
```

Listing 5.7 – Extrait de la configuration initiale dans `app.ts`

Architecture modulaire : Le dossier `src/routes/` Afin de ne pas surcharger le fichier `app.ts` et de respecter le principe de séparation des responsabilités, j'ai adopté une architecture modulaire. Chaque entité de l'application possède son propre fichier de routage dans le dossier `src/routes/`. Ces fichiers font le lien direct entre une URL spécifique et le contrôleur associé. Voici la liste exhaustive des routes développées :

- `adminRoutes.ts` : Gère les URL réservées à l'administration du système (gestion des codes, statuts, profils).
- `projectRoutes.ts` : Liste, création et détails des projets.
- `wpRoutes.ts` : Création et modification des lots de travaux (*Work Packages*) et génération depuis les templates.
- `taskRoutes.ts` : Ajout, assignation, modification et archivage des tâches.
- `userRoutes.ts` & `unitRoutes.ts` : Routes dédiées à la gestion des collaborateurs et de la structure de l'entreprise (départements).
- `preferenceRoutes.ts` : Enregistrement des préférences d'interface de l'utilisateur (comme l'affichage dynamique des colonnes des tableaux).
- `exportRoutes.ts` : Route unique déclenchant le téléchargement des données au format CSV.

Optimisation de l'ergonomie : l'implémentation des « Slugs » Au cours du développement, j'ai souhaité aller plus loin dans l'ergonomie de l'application en remplaçant les identifiants numériques classiques (IDs) dans les URLs par des **slugs**. Un « slug » est une version simplifiée et lisible d'un titre, optimisée pour les adresses web. Par exemple, au lieu d'accéder à un projet via l'URL `/project/14`, l'utilisateur y accède via `/project/yieldtrack-erp-v2`.

Cette découverte technique m'a permis d'améliorer deux aspects majeurs de l'application :

1. **La lisibilité** : L'utilisateur sait exactement où il se trouve et sur quel projet il travaille simplement en regardant la barre d'adresse de son navigateur.
2. **La robustesse du routage** : En utilisant des chaînes de caractères uniques, on évite d'exposer la structure interne incrémentale de la base de données.

L'implémentation a nécessité un travail coordonné entre le Frontend et le Backend. Pour automatiser ce processus et éviter toute erreur de saisie, j'ai intégré une logique JavaScript dans mes fenêtres modales de création et d'édition. Dès que l'utilisateur tape le nom d'un projet ou d'un lot de travaux, le slug est généré en temps réel grâce à une expression régulière

(*regex*) qui nettoie le texte :

```
1 // Ecouteur d'événement sur le champ "Nom"
2 document.getElementById('edit_wp_name').addEventListener('input',
3     function() {
4         const slug = this.value
5             .toLowerCase()
6             .normalize("NFD")           // Normalise les accents
7             .replace(/[\u0300-\u036f]/g, "") // Supprime les accents
8             .replace(/[^a-z0-9]/g, '-')    // Remplace tout ce qui
9             // n'est pas alphanumérique par un tiret
10            .replace(/-+/g, '-')           // Evite les tirets
11            // doubles
12            .replace(/^-|-$/g, '');        // Supprime les tirets
13            // au début et à la fin
14
15        document.getElementById('edit_wp_slug').value = slug;
16    });
```

Listing 5.8 – Algorithme de génération automatique de slug (edit-wp.ejs)

Une fois généré, ce slug est stocké dans la base de données via Sequelize. Côté serveur, j'ai ensuite adapté mes contrôleurs pour qu'ils ne cherchent plus l'élément par sa clé primaire (id), mais par son champ slug. Cette fonctionnalité, bien que demandant une rigueur supplémentaire dans la gestion des doublons, rapproche YieldTrack des standards des applications web modernes (SaaS).

Sécurisation des routes

Le concept de Middleware : Une révélation technique Dans l'écosystème Express, un **middleware** est une fonction logicielle qui s'intercale entre la réception de la requête HTTP et l'exécution du contrôleur. Il agit comme un poste de douane : il peut examiner la requête, la modifier, l'autoriser à passer (via la fonction `next()`), ou la rejeter en renvoyant une erreur.

D'un point de vue personnel, le concept de middleware m'avait semblé assez abstrait lors de mes cours à l'IUT. C'est véritablement en le mettant en pratique sur le projet YieldTrack que sa puissance et son utilité me sont apparues clairement. Il permet d'isoler la logique de sécurité et d'éviter de répéter du code de vérification au début de chaque fonction de contrôleur.

Authentification via JWT et gestion des Cookies Pour protéger mes fichiers dans le dossier `src/routes/`, j'ai basé ma sécurité sur les **JWT (JSON Web Tokens)** stockés dans des **cookies**. J'ai particulièrement apprécié travailler sur cette partie de gestion des cookies. C'est

une notion que j’avais déjà pu aborder en PHP par le passé, et la redécouvrir ici à travers le prisme de TypeScript et de Node.js (grâce à la librairie `cookie-parser`) a été techniquement très enrichissant.

Le principe est le suivant : lors de la connexion, le serveur génère un jeton (JWT) contenant l’identifiant de l’utilisateur et le place dans un cookie crypté sur son navigateur. À chaque nouvelle requête de l’utilisateur, les middlewares interceptent ce cookie pour vérifier son identité avant d’autoriser l’accès à une route.

```
1 export const requireAuth = (req: Request, res: Response, next:
  NextFunction) => {
2     const token = req.cookies.jwt; // Récupération du cookie
3
4     if (token) {
5         jwt.verify(token, process.env.JWT_SECRET as string,
            (err: any, decodedToken: any) => {
6             if (err) {
7                 res.redirect('/login'); // Jeton invalide ou
                    expiré
8             } else {
9                 req.user = decodedToken; // Stockage des infos
                    pour le contrôleur
10                next(); // Autorisation de passage vers le
                    contrôleur
11            }
12        });
13    } else {
14        res.redirect('/login'); // Aucun jeton trouvé
15    }
16 };
```

Listing 5.9 – Exemple de vérification du JWT dans `authMiddleware.ts`

Cartographie et rôles des Middlewares Afin de garantir une sécurité granulaire, j’ai développé plusieurs middlewares spécifiques, regroupés dans le dossier `src/middlewares/`, appliqués stratégiquement sur mes différentes routes :

- `authMiddleware.ts` : C’est le pare-feu principal. Il s’assure que l’utilisateur est bien connecté. S’il n’y a pas de jeton ou s’il est expiré, l’utilisateur est automatiquement redirigé vers la page de connexion. Il s’occupe également de rendre les informations de l’utilisateur courant accessibles à toutes les vues (*frontend*).

- `roleMiddleware.ts` : Ce middleware intervient juste après l'authentification. Il vérifie si l'utilisateur possède le profil requis pour effectuer une action. Par exemple, je l'applique sur le fichier `adminRoutes.ts` pour m'assurer que seul un profil "Administrateur" puisse y accéder.
- `projectMiddleware.ts` : Il gère une sécurité plus fine, liée au cœur de métier. Il vérifie les droits d'accès transversaux (par exemple, s'assurer qu'un utilisateur appartient bien à l'unité responsable d'un projet avant de lui permettre de modifier un *Work Package*).
- `preferencesMiddleware.ts` : Plus orienté "expérience utilisateur", ce middleware lit les cookies de préférences (non liés à la sécurité) pour déterminer par exemple quelles colonnes d'un tableau l'utilisateur avait choisi de masquer lors de sa précédente visite, afin de conserver son interface personnalisée.

Amélioration de l'expérience utilisateur : la redirection intelligente Au cours des tests de l'application, j'ai identifié un point de frustration ergonomique majeur : lorsqu'une session expire ou qu'un utilisateur non connecté tente d'accéder à un lien direct (par exemple, une fiche projet spécifique), il est systématiquement redirigé vers la page de connexion. Une fois identifié, l'utilisateur était renvoyé par défaut sur la page d'accueil, perdant ainsi sa cible initiale.

Pour remédier à cela et fluidifier la navigation, j'ai mis en place un mécanisme de "redirection vers la page d'origine". La logique se décompose en deux étapes techniques :

1. **La capture (Middleware) :** Lorsqu'un utilisateur tente d'accéder à une route protégée sans être connecté, mon `authMiddleware.ts` capture l'URL exacte qu'il demandait via la propriété `req.originalUrl`. Il le redirige ensuite vers la page de connexion en passant cette URL en paramètre dans l'adresse (*query parameter*).
2. **La redirection (Contrôleur) :** Lors de la soumission du formulaire de connexion, le contrôleur récupère ce paramètre. Si une destination était mémorisée, l'utilisateur y est renvoyé immédiatement après le succès de l'authentification ; sinon, il est dirigé vers l'accueil.

```

1 // Dans le Middleware (authMiddleware.ts)
2 if (!token) {
3     // On encode l'URL d'origine pour la passer en paramètre
4     const nextPath = encodeURIComponent(req.originalUrl);
5     return res.redirect('/login?next=${nextPath}');
6 }
7
8 // Dans le Contrôleur (authControllers.ts)

```

```

9 export const login = async (req: Request, res: Response) => {
10   const { email, password, next } = req.body; // 'next'
      provient du champ caché du formulaire
11
12   // ... Logique de vérification du mot de passe ...
13
14   // Redirection intelligente : vers la page demandée ou
      l'accueil par défaut
15   const redirectUrl = next && next !== 'undefined' ? next :
      '/';
16   res.redirect(redirectUrl);
17 };

```

Listing 5.10 – Mise en œuvre de la redirection dynamique (Middleware et Contrôleur)

Cette fonctionnalité, bien que simple en apparence, apporte une réelle valeur professionnelle à l'ERP. Elle permet aux collaborateurs de partager des liens directs vers des tâches ou des projets par e-mail ou via la barre d'adresse sans jamais perdre leur contexte de travail, même après une déconnexion.

Robustesse du serveur : Gestion des erreurs (Try/Catch) et redirection 404 Pour garantir la stabilité du serveur Node.js, une attention particulière a été portée à la gestion des erreurs. Contrairement à d'autres environnements (comme PHP), une exception non interceptée sous Node.js peut provoquer le crash complet de l'application, déconnectant ainsi tous les utilisateurs simultanément.

Pour prévenir ce risque critique, l'intégralité de la logique asynchrone (`async/await`) de mes contrôleurs est systématiquement encapsulée dans des blocs `try...catch`. Si une opération échoue en cours de route (par exemple, un problème de connexion à MariaDB ou une requête Sequelize mal formulée), l'exécution est immédiatement stoppée et bascule dans le bloc `catch`. Le serveur capture l'erreur silencieusement et renvoie un code HTTP approprié (comme 400 ou 500) accompagné d'un message d'erreur JSON compréhensible pour le Frontend, garantissant la continuité du service.

En parallèle des erreurs serveurs, il était impératif de gérer les erreurs de navigation côté client, communément appelées erreurs 404 (Page non trouvée). Ce cas se présente lorsqu'un utilisateur modifie manuellement l'URL ou clique sur un ancien lien brisé.

Dans le framework Express, les requêtes traversent les routes de manière séquentielle, de haut en bas. Pour gérer ce comportement de manière élégante, j'ai défini une route « attrape-tout » (wildcard `*`) placée à la toute fin de mon fichier `app.ts`, après toutes les autres déclarations de routes.

```

1 // ... Déclaration des routes valides (projects, tasks,
   users...) ...
2
3 // Route "Catch-all" pour les requêtes non interceptées (Erreur
   404)
4 app.use('*', (req: Request, res: Response) => {
5     res.status(404).render('404', {
6         user: req.user // Permet de garder la topbar avec le
           nom de l'utilisateur
7     });
8 });

```

Listing 5.11 – Gestion de l'erreur 404 en fin de routage dans app.ts

Ainsi, si une requête ne correspond à aucun contrôleur valide, elle finit sa course dans cette fonction de sécurité qui déclenche le rendu de la vue `404.ejs`. Cette page propose une interface amicale invitant l'utilisateur à retourner sur son tableau de bord, évitant ainsi le message d'erreur brut et peu esthétique généré par défaut par le navigateur.

5.2 Développement du Frontend

Bien que le développement d'interfaces graphiques ne soit pas mon domaine de prédilection, la création du *Frontend* est une étape incontournable : c'est la seule partie de l'application que l'utilisateur final verra et manipulera. Aujourd'hui, grâce aux outils modernes et aux moteurs de *templates*, cette tâche est grandement simplifiée et permet de générer des interfaces robustes et professionnelles très rapidement.

Pour garder un code propre et maintenable, j'ai structuré la partie visuelle de l'application de manière modulaire :

- `/public` : Contient les fichiers statiques accessibles directement par le navigateur (CSS, images, librairies JavaScript externes).
- `/views/Pages` : Contient les vues complètes de l'application (la page des projets, la page d'administration, etc.).
- `/views/modals` : Regroupe toutes les fenêtres superposées (pop-ups) permettant de créer ou modifier des données.
- `/views/partials` : Contient les "morceaux" de pages réutilisables (l'en-tête, le menu latéral, le pied de page).

Le moteur de rendu EJS : dynamiser le HTML Pour construire les pages, j’ai utilisé le moteur de *templates* **EJS** (*Embedded JavaScript*). Son rôle est crucial : il permet de transformer du code HTML statique en une interface dynamique en y injectant des données provenant directement du Backend.

L’utilisation d’EJS repose sur une syntaxe simple que j’ai rapidement prise en main :

- Les balises `<%= ... %>` permettent d’afficher une valeur (par exemple, le nom de l’utilisateur connecté dans la barre supérieure).
- Les balises `<% ... %>` permettent d’exécuter de la logique JavaScript (boucles `forEach`, conditions `if`) pour générer des éléments HTML à la volée, comme les lignes d’un menu ou les options d’une liste déroulante.

5.2.1 Le framework SB Admin 2

Pour garantir une ergonomie de qualité professionnelle sans avoir à réinventer la roue en CSS, je me suis appuyé sur **SB Admin 2**. Il s’agit d’un thème d’administration *open-source* basé sur le très populaire framework **Bootstrap 4**.

L’utilisation de ce *framework* m’a offert plusieurs avantages majeurs :

1. **Le système de grille (*Grid System*)** : Il permet de structurer la page en lignes et en colonnes de manière *responsive* (l’interface s’adapte automatiquement à la taille de l’écran).
2. **Des composants prêts à l’emploi** : SB Admin 2 fournit des cartes (*cards*), des boutons stylisés, des menus déroulants et des fenêtres modales avec un design moderne et unifié.
3. **Un gain de temps considérable** : J’ai pu me concentrer sur la logique de transmission des données plutôt que sur l’esthétique pure.

5.2.2 Réalisation des tableaux interactifs avec DataTables

L’affichage de grandes quantités de données (des centaines de tâches ou de lots de travaux) nécessitait un outil puissant. Un simple tableau HTML statique aurait rendu l’ERP inutilisable. J’ai donc intégré la bibliothèque JavaScript **DataTables.net**.

Cet outil transforme un tableau HTML standard en un composant interactif avancé offrant nativement : la pagination dynamique, une barre de recherche en temps réel, et le tri automatique.

Factorisation et maintenance : le composant `dynamic_table.ejs` Plutôt que de ré-écrire le code complexe des tableaux sur chaque page, j’ai appliqué le principe fondamental **DRY** (*Don’t Repeat Yourself*) en créant un composant unique et réutilisable : `dynamic_table.ejs`.

Ce fichier "partial" contient la structure générique du tableau et le menu de sélection dynamique des colonnes. En l’incluant simplement dans mes pages et en lui passant des paramètres (titre, URL de l’API), j’ai pu uniformiser toute l’interface de YieldTrack. Cette factorisation facilite grandement la maintenance : une modification sur ce fichier unique se répercute instantanément sur l’ensemble des tableaux de l’application.

```
1  const columnsDefinition = [  
2    { data: 'analyticalCode', name: 'analyticalCode', title:  
      'WBS' },  
3    {  
4      data: 'name', name: 'name', title: 'Project Name',  
5      render: function(data, type, row) {  
6        return '<a href="/project/${row.slug}">${data}</a>';  
7      }  
8    },  
9    {  
10     data: null, name: 'actions', title: 'Actions',  
11     orderable: false,  
12     render: function(data, type, row) {  
13       return '<button class="btn btn-link btn-sm"  
14         data-toggle="modal"  
15         data-target="#editProjectModal "  
16         data-id="${row.id}"><i class="fas fa-pen  
17         text-gray-400"></i></button>';  
18     }  
19   }  
20 ];  
21 // Appel de la fonction universelle définie dans footer.ejs  
22 initYieldtrackTable('projectsTable', '/project/api/data',  
23   columnsDefinition, prefs);
```

Listing 5.12 – Exemple d’initialisation d’un tableau via DataTables

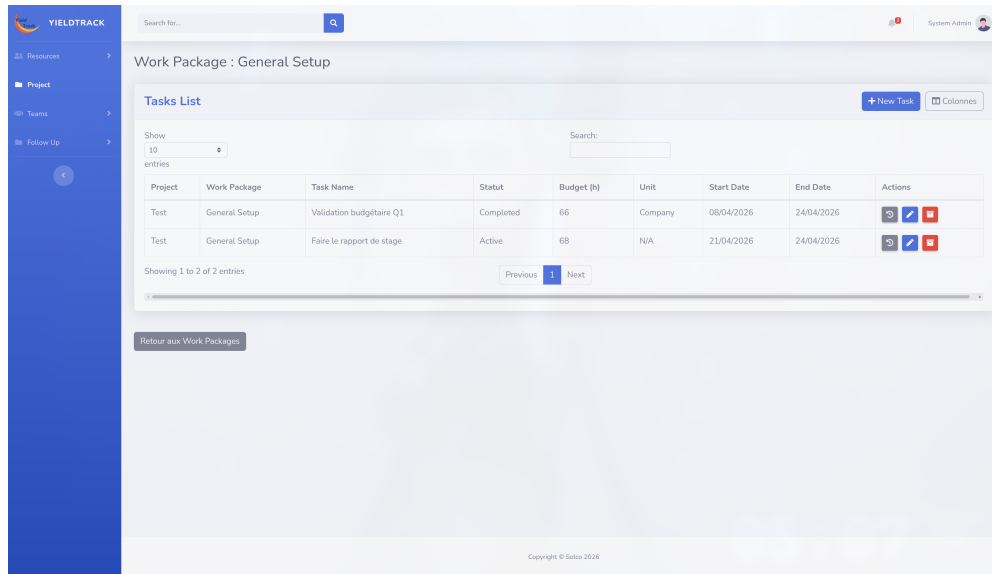


FIGURE 5.1 – Capture d’écran d’un tableau interactif (Pages listant les tâche du WorkPackage "General Setup" dans le projet "Test")

5.2.3 Pages, Modals et Validation : Le lien Frontend-Backend

L’ergonomie de l’application repose fortement sur l’utilisation des **Modals**. Au lieu de rediriger l’utilisateur vers une nouvelle page pour chaque création, un formulaire s’ouvre par-dessus le tableau actuel. C’est ici qu’intervient la **double validation des données** :

1. La validation Frontend : J’ai utilisé les attributs HTML5 natifs (required, pattern, type="email") pour empêcher l’envoi d’un formulaire vide ou mal formaté directement dans le navigateur.

2. La validation Backend et l’interaction AJAX (Fetch API) : Une fois le formulaire soumis, j’utilise la Fetch API pour envoyer les informations en arrière-plan au contrôleur. Si le Backend détecte une erreur (par exemple, un e-mail déjà existant), mon script intercepte la réponse et affiche une alerte directement dans la *Modal*, sans que l’utilisateur ne perde sa saisie :

```

1 form.addEventListener('submit', async function(e) {
2   e.preventDefault();
3   const alertContainer =
4     document.getElementById('alert-container');
5   const data = Object.fromEntries(new
6     FormData(this).entries());
7
8   try {
9     const response = await fetch(this.action, {

```

```

8         method: 'POST',
9         headers: { 'Content-Type': 'application/json' },
10        body: JSON.stringify(data)
11    });
12    const result = await response.json();
13    if (response.ok) {
14        window.location.reload();
15    } else {
16        alertContainer.innerHTML = `<div class="alert
17                                alert-danger m-3">${result.error}</div>`;
18    }
19    } catch (err) {
20        alertContainer.innerHTML = `<div class="alert
21                                alert-danger">Server connection error.</div>`;
22    }
23    });

```

Listing 5.13 – Gestion asynchrone des erreurs de saisie (Fetch API)

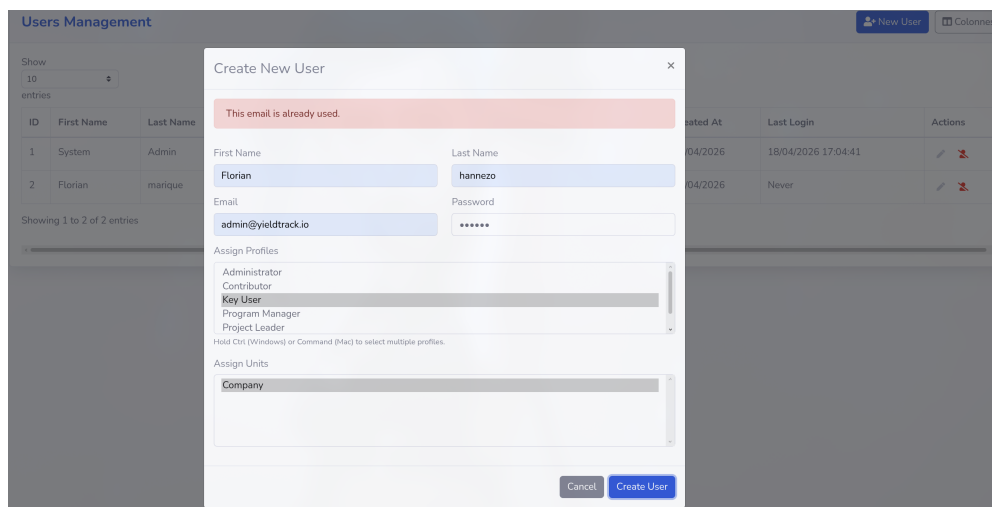


FIGURE 5.2 – Retour d’erreur dynamique dans une fenêtre modale (Création d’un user avec un e-mail déjà existant)

AJAX ou Asynchronous JavaScript and XML, est une technique de développement Web dans laquelle une application Web récupère le contenu du serveur en effectuant des requêtes HTTP asynchrones et utilise le nouveau contenu pour mettre à jour les parties pertinentes de la page sans nécessiter un chargement complet de la page.

Cartographie exhaustive des Vues (Frontend)

Afin de documenter la portée de l’interface graphique développée, voici la liste exhaustive des fichiers EJS constituant les écrans et composants de YieldTrack :

Les Pages Principales (/views/Pages/) :

- login.ejs & reset-password.ejs : Écrans d’authentification et de sécurité.
- projects.ejs & projectsDetails.ejs : Tableaux de bord de gestion du WBS.
- wpDetails.ejs : Vue détaillée d’un lot de travaux et de ses tâches.
- remaining.ejs : Interface de saisie du *Remaining Work*.
- allocation.ejs : Interface de gestion des équipes.
- users.ejs, adminUnits.ejs, adminCodes.ejs : Pages d’administration.
- export.ejs : Interface d’exportation de données CSV.
- 404.ejs : Page d’erreur personnalisée.

Les Fenêtres Modales (/views/modals/) :

- *Gestion de Projet* : projects.ejs, edit-project.ejs, wp-custom.ejs, wp-template.ejs, edit-wp.ejs.
- *Gestion des Tâches* : task.ejs, edit-task.ejs, archive-task.ejs.
- *Suivi du Temps* : add-rw.ejs, rw-history.ejs.
- *Administration* : create-user.ejs, edit-user.ejs, create-unit.ejs, assign-user-unit.ejs.

Les Composants Partagés (/views/partials/) :

- sidebar.ejs, topbar.ejs, header.ejs, footer.ejs : Éléments de structure communs.
- dynamic_table.ejs : Squelette générique des tableaux interactifs.

5.3 Déploiement sur un VPS

Le déploiement de l’application sur un serveur de production a été la phase la plus intense et, paradoxalement, la plus frustrante de mon stage. L’objectif était de mettre YieldTrack en ligne sur un **VPS** (*Virtual Private Server* ou Serveur Privé Virtuel) loué chez HostPapa.

D’un point de vue technique, un VPS est une machine virtuelle qui offre à l’utilisateur le même niveau d’accès et de contrôle (droits administrateur ou *root*) qu’un véritable ser-

veur physique dédié, mais dont les ressources matérielles sont isolées et partagées sur une machine hôte. Contrairement à un hébergement web classique (mutualisé), le VPS permet d'installer et de configurer librement l'environnement logiciel complet de son choix (système d'exploitation Linux, serveurs web, Node.js, MariaDB).

Cependant, cette liberté s'accompagne d'une grande complexité. Cette phase a été marquée par des contraintes de temps majeures : HostPapa ne nous a fourni les accès au VPS que le jeudi 9 avril, soit la veille de la fin officielle de mon stage. Disposant de quelques heures pour tout mettre en ligne, je me suis retrouvé confronté à un environnement **cPanel** que je ne connaissais pas, aux antipodes de mes expériences sur les VPS "nus" d'OVH ou les serveurs locaux vus à l'IUT.

Face à cette urgence, j'ai rencontré une cascade de problèmes de configuration réseau et système :

1. Conflit de serveurs web (Nginx vs Apache sur cPanel) J'ai initialement tenté d'installer Nginx manuellement pour gérer le *reverse proxy* de mon application. Cela a provoqué un échec immédiat. Sur ce VPS, le port 80 (HTTP) était déjà occupé par Apache (le serveur par défaut de cPanel), qui intégrait sa propre couche Nginx invisible. Lancer un Nginx indépendant créait un conflit de port fatal. Il a fallu comprendre qu'il fallait utiliser Apache pour relayer les requêtes vers Node.js.

2. Architecture des dossiers et Sécurité (Le piège de /root) Dans la précipitation, j'ai placé le code source dans le dossier super-administrateur (/root/), puis directement dans le dossier public (public_html). Mettre le code dans public_html exposait le fichier .env contenant les mots de passe. À l'inverse, le placer dans /root/ empêchait Apache d'y accéder par sécurité. Le code a dû être isolé dans /home/yieldtrack/yieldtrack-test/.

3. Problème de droits et de permissions (Erreur 403 Forbidden) J'ai créé des fichiers de test via SSH en tant que root. Le serveur Apache (qui tourne sous un utilisateur restreint) s'est vu refuser l'accès en lecture par le système Linux, provoquant une erreur "403 Forbidden". J'ai dû réattribuer la propriété des dossiers via la commande `chown`.

4. La restriction du .htaccess pour les Proxys Pour relier le domaine au port de mon serveur Node.js, j'ai tenté d'utiliser la directive Proxy ([P]) dans le fichier .htaccess. Cela a généré une nouvelle erreur 403, car cPanel bloque souvent la création de tunnels proxy via ce fichier pour éviter les failles de sécurité. Il a fallu contourner cela via l'interface graphique "Setup Node.js App".

5. Confusion entre lignes de commande et fichiers de configuration Toujours dans l'urgence, j'ai collé des directives Apache directement dans mon terminal (Bash), ce qui m'a

logiquement retourné une erreur de type *command not found*. J'ai dû apprendre à différencier le *shell* des fichiers de configuration édités via nano.

6. La configuration DNS (EuroDNS) La toute dernière étape consistait à faire pointer le nom de domaine (EuroDNS) vers l'IP du VPS. L'interface d'EuroDNS exigeait de renseigner quatre serveurs de noms (*nameservers*) pour garantir la redondance, ce qui bloquait la propagation classique à deux adresses que je connaissais.

Bilan de cette étape : La limite du stage Contrairement au développement de l'application où chaque erreur (SQL, TypeScript) était formatrice et compréhensible, cette phase de déploiement m'a laissé un sentiment de frustration. Malgré l'ouverture de tickets de support technique auprès d'HostPapa et l'utilisation intensive de l'intelligence artificielle pour tenter de débloquer la situation, je n'ai pas réussi à rendre le serveur pleinement opérationnel par mes propres moyens. J'ai dû me résoudre à arrêter mes recherches d'infrastructure le jeudi soir à 00h30 (soit le vendredi matin de mon dernier jour).

C'est à ce moment précis que je me suis heurté à la limite majeure de mon stage : **l'absence d'un développeur Senior ou d'un administrateur système**. Être seul face à une "boîte noire" (cPanel, Nginx/Apache imbriqués, pm2) dans l'urgence m'a fait perdre un temps précieux. Un encadrement technique m'aurait permis de comprendre la logique globale de l'infrastructure plutôt que d'appliquer des correctifs à l'aveugle proposés par l'IA.

Bien que le site ait finalement pu être mis en ligne la semaine suivante, grâce notamment à mon tuteur qui a pris le relais sur la gestion des *nameservers* (DNS), la maîtrise complète et sereine d'un environnement de production complexe reste pour moi une zone d'ombre que j'aurai à cœur d'éclaircir dans la suite de mon parcours professionnel.

5.4 Phase de tests et validation des flux métiers

Dans le cadre du développement de YieldTrack, la stratégie de test a dû être adaptée aux contraintes de temps imposées par la durée du stage (huit semaines). La mise en place de tests unitaires automatisés (via des bibliothèques comme *Jest* ou *Mocha*) aurait nécessité un investissement chronophage au détriment du développement des fonctionnalités principales. J'ai donc fait le choix stratégique de privilégier des **tests fonctionnels et d'intégration basés sur les flux métiers**.

Étant donné que l'application est un ERP dont la logique repose entièrement sur des rôles stricts, tester une fonction isolée n'avait que peu de sens. Il était crucial de s'assurer que les données transitaient correctement d'un profil à un autre.

Pour réaliser ces tests de bout en bout (*End-to-End*), je me suis appuyé sur le fichier d'insertion de données fictives (`data.sql`) créé au début du projet. J'ai simulé des scénarios d'utilisation réels en ouvrant plusieurs navigateurs simultanément (ou des fenêtres de navigation privée) pour incarner différents acteurs de l'entreprise :

1. **Étape 1 (Program Manager)** : Connexion pour créer un nouveau projet et générer des lots de travaux à partir d'un *Corporate Template*.
2. **Étape 2 (Project Leader)** : Connexion pour valider l'arborescence et créer des tâches spécifiques, en les attribuant à une unité.
3. **Étape 3 (Team Manager)** : Connexion sur la page d'allocation pour assigner la tâche fraîchement créée à un contributeur précis de son équipe.
4. **Étape 4 (Contributeur)** : Connexion finale pour déclarer une estimation de temps (*Remaining Work*) sur cette tâche.

Cette méthodologie de test "en conditions réelles" s'est révélée très efficace. C'est d'ailleurs en réalisant ces parcours utilisateurs croisés que j'ai pu identifier certaines incohérences ergonomiques (qui ont conduit à la création de la redirection intelligente détaillée précédemment) et affiner la gestion des droits d'accès. Bien que manuelle, cette validation par les flux garantit que l'application livrée répond de manière cohérente aux véritables processus de l'entreprise Solco.

5.5 Documentation

La documentation est la clé de voûte de tout projet informatique logiciel, garantissant sa pérennité, sa maintenabilité et sa transmission. Initialement, cette phase devait être réalisée en binôme avec ma collègue pour croiser nos regards et tester mutuellement nos tutoriels. Malheureusement, son absence pour cause de maladie lors de ma dernière semaine m'a contraint à finaliser ce travail de manière autonome. Bien que le contenu soit présent, cette situation a limité la phase de relecture externe, pourtant cruciale pour valider la clarté d'une documentation technique.

5.5.1 Listing des fonctions et de leurs objectifs

Afin de faciliter la reprise du code par un futur développeur, j'ai rédigé un document d'architecture détaillé. Ce livrable cartographie l'ensemble de l'application et explique le couplage fort entre le Backend (Node.js/Express) et le Frontend (EJS/DataTables).

Il documente de manière exhaustive chaque module (Projets, Work Packages, Tâches,

Utilisateurs), en listant les fonctions des contrôleurs, les routes API associées, et les structures de données (comme la définition centralisée des colonnes via des constantes). Il détaille également le flux de données, de la requête AJAX de l'utilisateur jusqu'au renvoi du JSON par le serveur, agissant comme un véritable mode d'emploi de l'architecture logicielle.

5.5.2 Développement en local

Pour permettre l'intégration de nouveaux collaborateurs ou la préparation de l'ERP pour de nouveaux clients, j'ai conçu un guide pas-à-pas de déploiement local compatible Windows et Mac.

Ce tutoriel vulgarise l'installation des prérequis techniques (VS Code, Node.js, XAMPP, Git) et guide l'utilisateur à travers la récupération du code source, la configuration de la base de données MariaDB locale, et la sécurisation des variables via le fichier `.env`. Bien que ce guide soit pensé pour être le plus accessible possible, l'absence de ma collègue n'a pas permis de le faire "tester à l'aveugle" par un œil neuf pour valider sa totale intelligibilité.

5.5.3 Déploiement de l'application sur un serveur

La dernière partie de la documentation concerne la mise en production. J'ai rédigé un guide expliquant la connexion SSH à un VPS sous AlmaLinux, l'installation de Nginx comme *Reverse Proxy* (un serveur intermédiaire qui intercepte les requêtes du web pour les rediriger vers ton serveur Node.js interne), la gestion des processus avec pm2, et la mise en place d'une automatisation CI/CD (Continuous Integration / Continuous Deployment), avec GitHub Actions qui sert à automatiser les tests et la mise en ligne.

Cependant, en toute transparence, ce document met en lumière la limite majeure de mon travail de déploiement abordée dans la section précédente. Ayant été rédigé théoriquement avant ma confrontation au véritable VPS HostPapa et à son environnement **cPanel**, ce tutoriel décrit un déploiement "idéal" sur un serveur nu. Dans le contexte réel de Solco, une grande partie de ces instructions (notamment sur l'installation de Nginx ou la gestion manuelle des ports) s'est révélée obsolète ou bloquée par les restrictions de cPanel. Ce décalage confirme qu'une documentation d'infrastructure ne peut être valide que si l'auteur maîtrise pleinement l'environnement technique de destination.

Bilan personnel et professionnel

6.1 Bilan professionnel : De la théorie à la pratique

Sur le plan technique, ces huit semaines de stage au sein de Solco ont été extrêmement formatrices. Arrivé avec des connaissances solide, j'ai su m'adapter rapidement à un environnement de production moderne.

Ce stage m'a poussé à faire preuve d'une grande autonomie. J'ai pu découvrir, apprendre et implémenter par moi-même un ensemble de technologies que je ne maîtrisais pas auparavant : le typage strict avec **TypeScript**, la manipulation d'une base de données via un ORM (**Sequelize**), le maintien d'applications avec **PM2**, et les bases de la gestion d'un nom de domaine.

Cependant, cette montée en compétences rapide n'aurait pas été possible sans les fondations acquises à l'IUT Nancy Charlemagne. Les concepts d'architecture logicielle vus en cours, notamment à travers le langage PHP (le modèle MVC, la gestion des sessions et des cookies), m'ont servi de socle. Ils m'ont permis de comprendre la logique des outils JavaScript que j'utilisais, transformant l'apprentissage en une transition logique.

Néanmoins, cette expérience m'a aussi confronté à la réalité du métier. Le manque de temps, exacerbé par la livraison tardive du serveur VPS, ainsi que l'absence d'un développeur *Senior* pour m'épauler sur l'infrastructure cPanel, ont généré une frustration certaine. J'ai pris conscience que dans un environnement de production complexe, le mentorat technique est un levier essentiel pour transformer des correctifs appliqués à l'aveugle en une véritable compréhension système.

6.2 Bilan personnel : Donner du sens au code

Sur le plan humain, ce stage a été une expérience profondément plaisante. L'accueil de l'équipe de Solco et la confiance accordée par M. Régis Marique m'ont permis de travailler dans un cadre stimulant au Luxembourg.

La plus grande leçon de ce stage s'est déroulée lors de ma visite sur le terrain chez le client SEO. Avant cette immersion, je percevais YieldTrack comme un ensemble de relations et de lignes de code. En observant les collaborateurs utiliser des fichiers Excel rigides pour saisir leur « reste à faire », j'ai eu un déclic. J'ai compris l'utilité réelle de l'outil que je construisais.

Donner du sens à mon travail est une source de motivation primordiale ; cette rencontre m'a fait réaliser qu'être développeur, c'est avant tout concevoir des solutions pour résoudre des problèmes humains concrets.

6.3 Adéquation avec les compétences du BUT Informatique

Ce stage s'inscrit en parfaite adéquation avec le référentiel de compétences de deuxième année de l'IUT Nancy Charlemagne :

- **Réaliser des applications** : J'ai conçu l'ERP YieldTrack en utilisant des patterns usuels (MVC) et en analysant les besoins métiers (logic RW).
- **Gérer des données** : J'ai modélisé une base relationnelle complexe et utilisé SQL et un ORM pour garantir l'intégrité des informations.
- **Administrer les applications** : Malgré les difficultés, j'ai configuré un serveur VPS, géré des services réseau (HTTP, DNS) et utilisé des outils comme PM2 pour maintenir l'application en ligne.
- **Conduire un développement et Collaborer** : La visite chez SEO pour le recueil des besoins et l'utilisation de Git pour le versioning valident ma capacité à travailler en projet et en équipe.

Conclusion

Au terme de ces huit semaines de stage au sein de l'entreprise Solco, le bilan s'avère extrêmement positif, tant sur le plan technique que personnel. La mission qui m'a été confiée — le développement « from scratch » de l'application ERP YieldTrack — représentait un défi de taille pour un étudiant de deuxième année. Elle m'a permis de traverser l'intégralité du cycle de vie d'un projet logiciel professionnel : depuis la modélisation de la base de données MariaDB jusqu'au déploiement final sur un serveur de production, en passant par l'orchestration complexe d'un Backend en Node.js et TypeScript.

D'un point de vue technique, ce projet m'a permis une montée en compétences accélérée. J'ai pu transformer les bases théoriques acquises à l'IUT en outils concrets, en apprenant à maîtriser des technologies de pointe comme l'ORM Sequelize pour la gestion des données relationnelles ou le framework Express pour le routage. Bien que la phase de déploiement sur le VPS HostPapa ait révélé des zones d'ombre dues à mon manque d'expérience en infrastructure et à l'absence de mentorat senior dans ce domaine, ces difficultés n'ont fait que renforcer ma détermination à approfondir la culture DevOps (l'unification du développement et de l'administration système). J'ai appris qu'un développeur ne doit pas seulement savoir écrire du code, mais aussi comprendre l'environnement complexe dans lequel son application évolue.

Sur le plan humain, ce stage a donné un véritable sens à mon parcours. La visite chez le client SEO a été le moment le plus marquant de mon immersion : elle a transformé ma perception de l'informatique. J'ai compris que derrière chaque ligne de code se cache une solution destinée à simplifier la vie d'un utilisateur. Développer YieldTrack n'était plus seulement un exercice de programmation, mais une mission visant à offrir à Solco et à ses partenaires un outil de pilotage plus agile, plus fiable et plus performant que les anciens systèmes basés sur Excel.

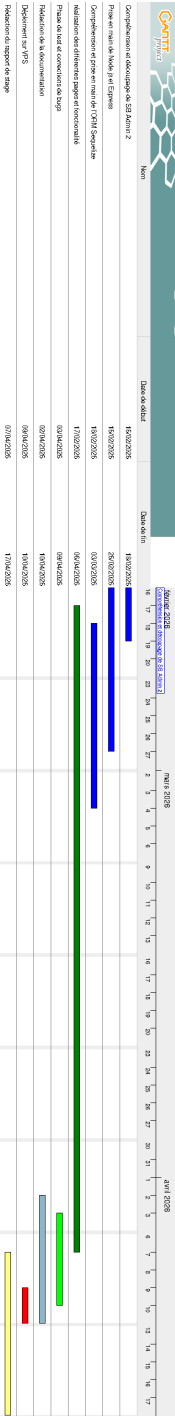
Enfin, cette expérience m'a permis de m'intégrer pleinement dans la culture de l'entreprise Solco, dont je partage les valeurs de simplicité et d'efficacité. Je suis particulièrement fier d'avoir pu livrer une application fonctionnelle et en ligne avant la fin de mon stage. Cependant, je suis conscient que YieldTrack est un outil vivant qui ne demande qu'à évoluer : l'amélioration de la délivrabilité des e-mails, l'optimisation fine des performances du serveur ou encore l'ajout de nouvelles fonctionnalités métier sont autant de pistes passionnantes pour l'avenir.

C'est avec une grande motivation que je me déclare prêt et enthousiaste à l'idée de retravailler avec Solco dans le futur. Que ce soit dans le cadre d'une alternance ou d'une collaboration future, je serais ravi de continuer à perfectionner YieldTrack, d'en éclaircir les dernières zones d'ombre techniques et d'accompagner l'entreprise dans sa transformation numérique. Ce stage n'est pas seulement la fin d'un cursus de deuxième année, mais le début d'une collaboration que j'espère longue et fructueuse avec Régis Marique et son équipe.

Bibliographie

- [cPanel, 2026] cPanel, *How to Install a Node.js Application*, consulté le 9 avril 2026, <https://docs.cpanel.net/knowledge-base/web-services/how-to-install-a-node.js-application/>
- [DataTables, 2026] DataTables, *DataTables Manual*, consulté le 25 février 2026, <https://datatables.net/manual/>
- [DigitalOcean, 2020] DigitalOcean, ONYEKWE Kelechi, *How To Authenticate Node.js Applications with JSON Web Tokens*, consulté le 4 mars 2026, <https://www.digitalocean.com/community/tutorials/nodejs-jwt-expressjs>
- [Express.js, 2026] Express.js, *Using Express middleware*, consulté le 20 février 2026, <https://expressjs.com/en/guide/using-middleware.html>
- [MDN Web Docs, 2014] MDN Web Docs, *String.prototype.replace()*, consulté le 15 mars 2026, https://developer.mozilla.org/fr/docs/Web/JavaScript/Reference/Global_Objects/String/replace
- [Nodemailer, 2026] Nodemailer, *Nodemailer documentation*, consulté le 28 mars 2026, <https://nodemailer.com/about/>
- [Sequelize, 2026] Sequelize, *Manual | Sequelize - TypeScript*, consulté le 22 février 2026, <https://sequelize.org/docs/v6/other-topics/typescript/>
- [Stack Overflow, 2015] Stack Overflow, *How to include nested associations in Sequelize*, consulté le 28 février 2026, <https://stackoverflow.com/questions/33215262/how-to-include-nested-associations-in-sequelize>

Diagramme de Gantt



Historique détaillé de l'entreprise Solco

Date / Période	Événement et Évolution
Début 2025	Création de Solco par M. Régis Marique. La société est un <i>spin-off</i> d'une ancienne structure de conseil, permettant de conserver une expertise métier immédiate.
Courant 2025	Implantation stratégique au Luxembourg (Livange), point central pour collaborer avec les marchés français et belges.
2025 - 2026	Phase de consolidation : récupération d'anciens collaborateurs et signature des trois premiers contrats majeurs dans la région FranBenelux.
Début 2026	Phase de croissance : lancement des sessions de recrutement et décision de développer l'application propriétaire Yield-track pour remplacer les outils de suivi Excel.

TABLE B.1 – Historique chronologique de la société Solco

Fiche bibliothèque

Auteur : Florian HANNEZO

Titre : Développement et mise en production d'une application de gestion de tâches hiérisées

Entreprise : Solco

Année universitaire : 2025–2026

Diplôme : DUT / BUT informatique – IUT Nancy Charlemagne

Mots-clés : Développement web, ERP, Node.js, TypeScript, MariaDB, Sequelize, Architecture MVC, Gestion de projet.

Résumé (synopsis) :

Ce rapport retrace la conception intégrale (from scratch) de YieldTrack, un ERP simplifié destiné à optimiser le suivi de projet par le "Reste à faire" (Remaining Work). Le travail réalisé couvre la modélisation d'une base de données relationnelle, le développement d'un backend robuste sous Node.js/TypeScript avec l'ORM Sequelize, et la mise en place d'une interface dynamique. Une phase de déploiement sur VPS vient compléter cette immersion technique au cœur des enjeux de transformation numérique d'un cabinet de conseil.